



Today's agenda

↳ BST

↳ Search in BST

↳ isBST

↳ insert in BST

↳ Recover BST



AlgoPrep

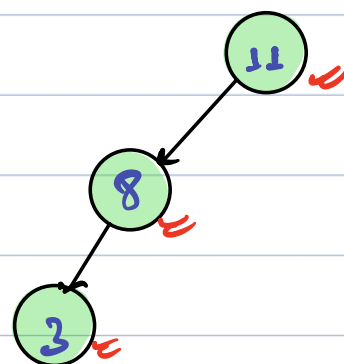
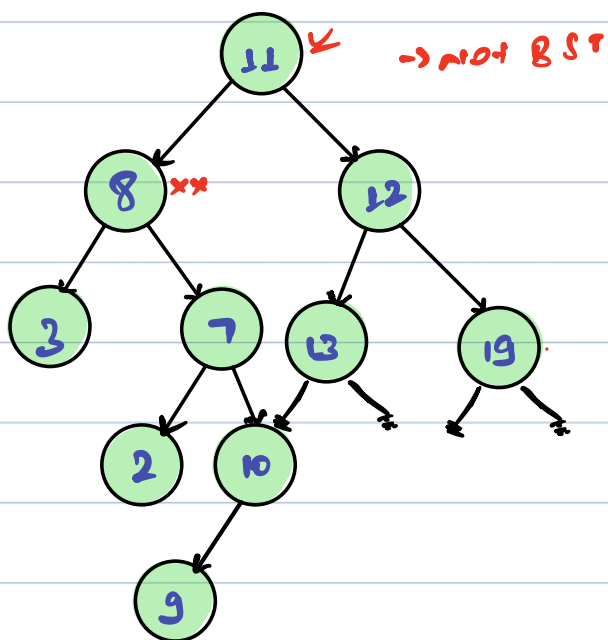
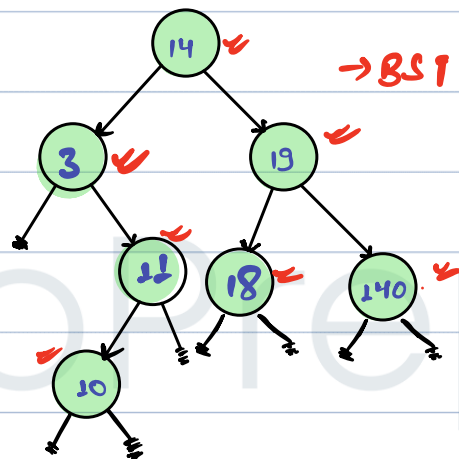
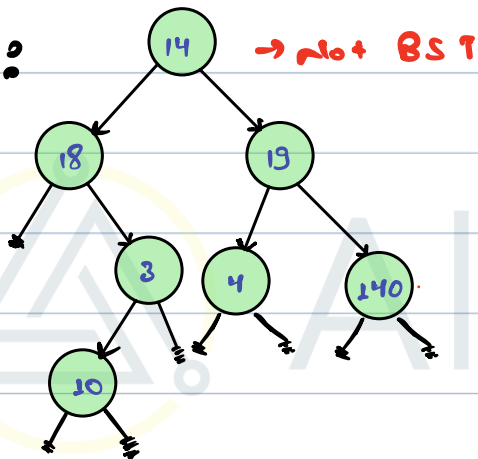


BST: Binary Search tree

↳ A Binary Tree is a BST if:

for all nodes: All the nodes in the left subtree $<$ Node.val $<$ All the nodes in the right subtree

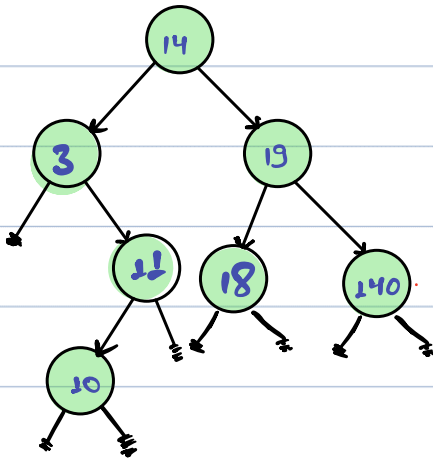
Ex 1:





Property:

↳ Inorder in BST is always sorted.



→ 3 10 11 14 18 19 140

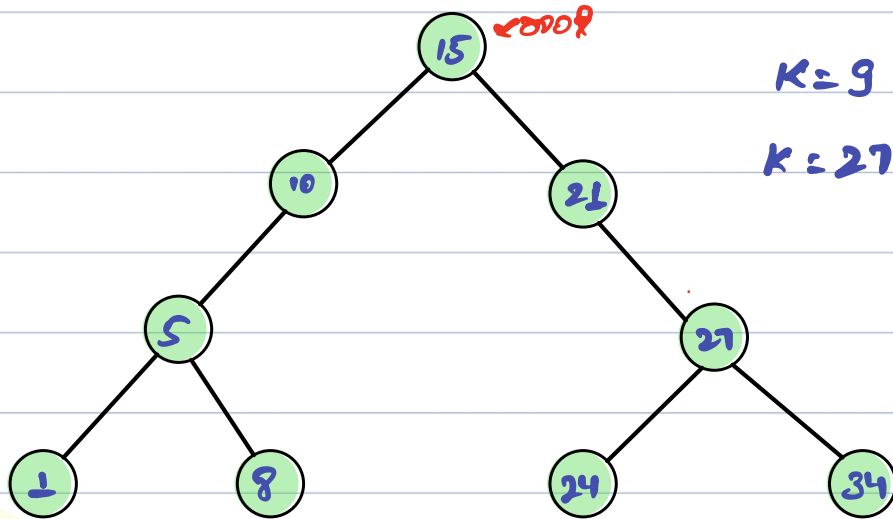
↳ LNR

Note: BST in general Can't have duplicate values.



Q) Search K in BST

↳ Given a BST and a value K , return true if a Node with value K is Present else false.



$K = 9 \rightarrow \text{false}$

$K = 27 \rightarrow \text{true}$

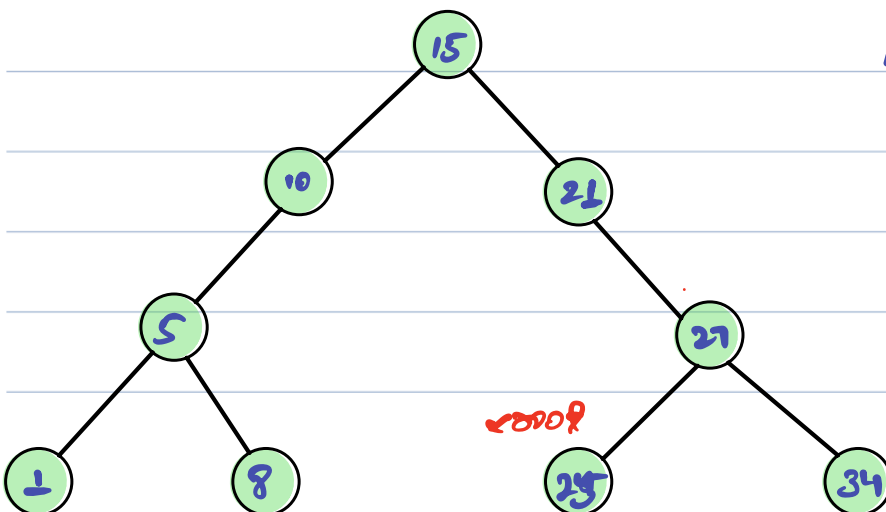
Idea 1

↳ write any traversal and check if K is Present?

T.C: $O(N)$

S.C: $O(N)$

Idea 2



$K = 24$

$K > 15 \rightarrow \text{go to right}$

$K > 21 \rightarrow \text{go to right}$

$K < 27 \rightarrow \text{go to left}$

$K == 24 \rightarrow \text{return true}$



// Pseudo code

```
boolean search (Node root, int k) {
```

```
    while (root != null) {
```

```
        if (root.val == k) {
```

```
            return true;
```

```
        }
```

```
        else if (root.val > k) {
```

```
            root = root.left;
```

```
        }
```

```
        else {
```

```
            root = root.right;
```

```
        }
```

```
    }
```

```
    return false;
```

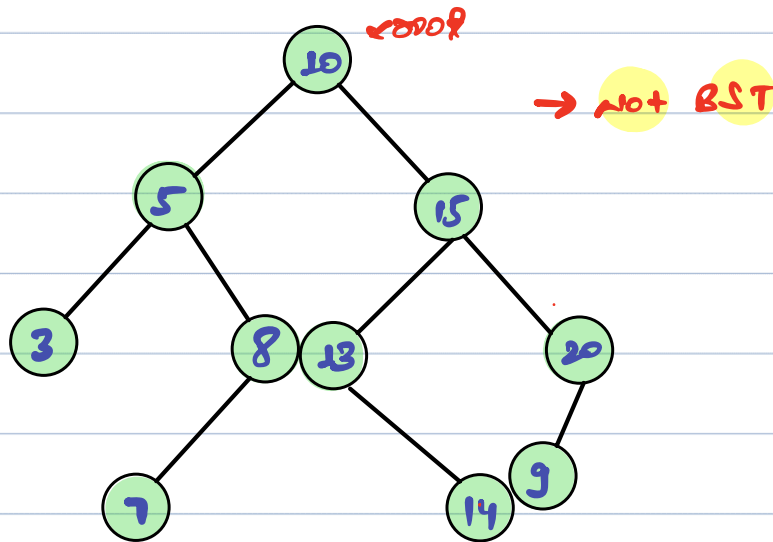
```
}
```

T.C: $O(H)$ $\rightarrow O(\log n)$
S.C: $O(1)$



Q) isBST?

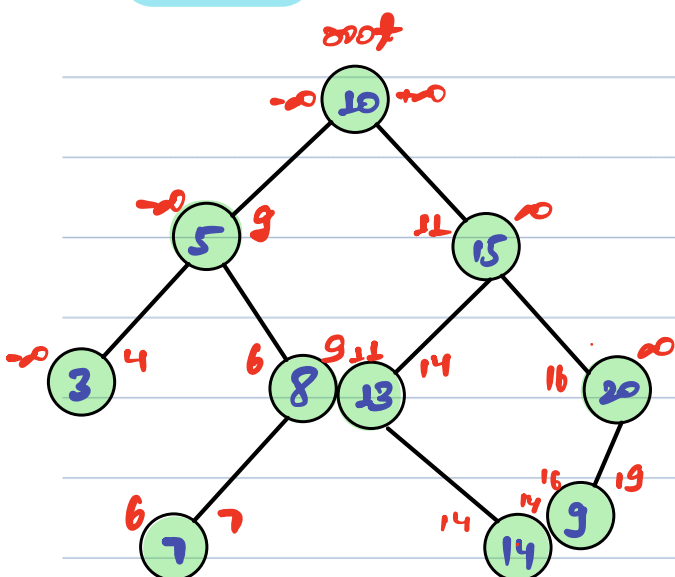
↳ Given BT, Check if it is a BST or Not?



11 idea 1

- ↳ ① Do inorder and store all the elements in array.
- ↳ ② Check whether array is sorted or not?

11 idea 2



min = $-\infty$

max = $+\infty$

going to left: { par.min, par.val }
going to right: { par.val, par.max }



// Pseudo Code

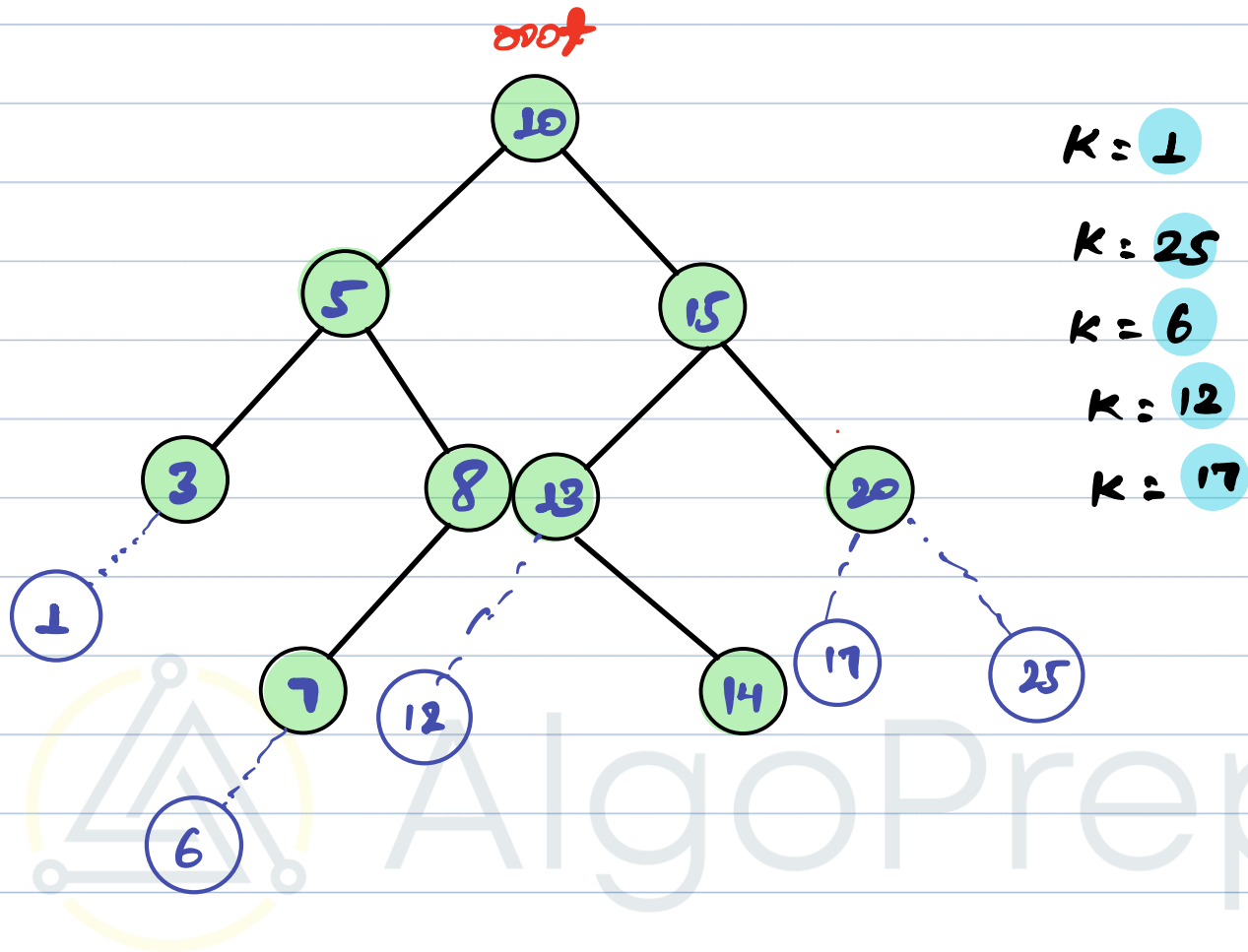
```
boolean isBST (Node root, int min, int max) {  
    if (root == null) { return true; }  
    if (root.val < min || root.val > max) {  
        return false;  
    }  
    boolean l = isBST (root.left, min, root.val - 1);  
    boolean r = isBST (root.right, root.val + 1, max);  
    if (l == false || r == false) {  
        return false;  
    }  
    return true;  
}
```

T.C: $O(N)$

S.C: $O(1)$



Q) insert a node with value K in a BST.



↳ you can always add as a child node of a leaf node.



// Pseudo code

```
void insert (Node root, int k) {  
    Node n = new Node (k);  
    while (true) {  
        if (root.val > k) {  
            if (root.left == null) {  
                root.left = n;  
                return;  
            } else {  
                root = root.left;  
            }  
        } else {  
            if (root.right == null) {  
                root.right = n;  
                return;  
            } else {  
                root = root.right;  
            }  
        }  
    }  
}
```

T.C: $O(H)$

S.C: $O(1)$



Q) Recover sorted arr[]

↳ Given an arr[] which is formed by swapping 2 distinct index in a sorted {inc} arr: {1 swap}

Ex: arr[] = { 4 10 19 14 18 12 25 28 }

→ Compare Consecutive indexes.

↳ 1st failed Condition: ith index is the faulty one.

↳ 2nd failed Condition: (i+1)th index is the faulty one.

Algo

arr[] = { 4 10 19 14 18 12 25 28 }

ansidx1 = ~~2~~

ansidx2 = ~~5~~

edge case:

arr[] = { 3 6 10 15 12 17 20 33 }

ansidx1 = ~~3~~

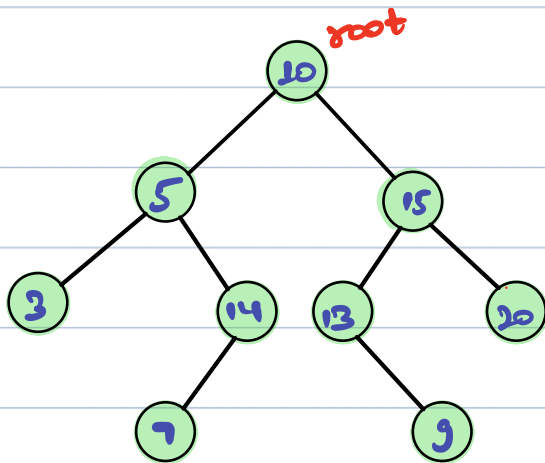
ansidx2 = ~~4~~



Q) Recover BST

↳ Given a BST, which is formed by swapping 2 distinct nodes, recover original BST.

↳ inorder of BST is sorted.



Node f;
Node s;
Node Prev;

```
void inorder (Node curr) {  
    if (curr == null) { return; }
```

```
    inorder (curr.left);
```

```
    if (Prev != null && curr.val < Prev.val && f == null) {
```

```
        f = Prev;  
        s = curr;
```

```
    } else if (Prev != null && curr.val < Prev.val && f != null) {  
        s = curr;
```

```
    }  
    Prev = curr;
```

```
    inorder (curr.right);  
}
```

T.C: $O(N)$

S.C: $O(H)$

Pre: ~~3~~ ~~5~~ ~~7~~ ~~14~~ 10

b = ~~14~~

s: ~~10~~



void inorder (Node curr)

1 if (curr == null) { return; }

2 inorder (curr.left);

if (prev != null && curr.val < prev.val && b == null)

b = prev;
s = curr;

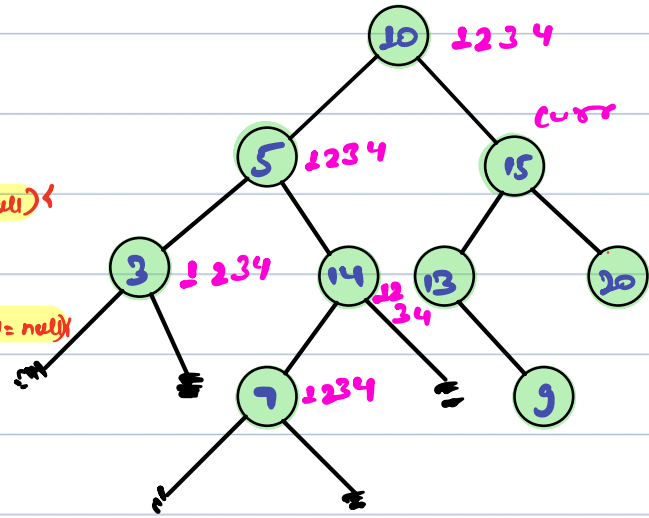
3 else if (prev != null && curr.val < prev.val && b != null)

s = curr;

prev = curr;

4 inorder (curr.right);

3



3 5 7 14 10



AlgoPrep